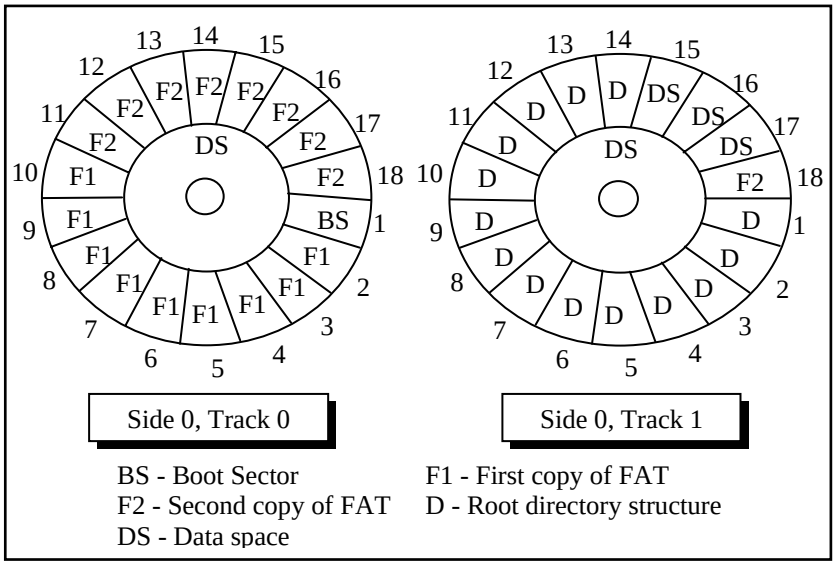

Figure 19.5

With the logical structure of the floppy disk behind us let us now write a program that reads the boot sector of a floppy disk and displays its contents on the screen. But why on earth would we ever like to do this? Well, that's what all Windows-based Anti-viral softwares do when they scan for boot sector viruses. A good enough reason for us to add the capability to read a boot sector to our knowledge! Here is the program…

```
# include <stdafx.h>
# include <windows.h>
# include <stdio.h>
# include <conio.h>

# pragma pack ( 1 )
struct boot
{
     BYTE jump [ 3 ] ;
```

```
    char bsOemName [ 8 ] ;
    WORD bytesperSector ;
    BYTE sectorspercluster ;
    WORD sectorsreservedarea ;
    BYTE copiesFAT ;
    WORD maxrootdirentries ;
    WORD totalSectors ;
    BYTE mediaDescriptor ;
    WORD sectorsperFAT ;
    WORD sectorsperTrack ;
    WORD sides ;
    WORD hiddenSectors ;
    char reserve [ 480 ] ;
} ;

void ReadSector ( char*src, int ss, int num, void* buff  ) ;

void main( )
{
    struct boot b ;
    ReadSector ( "\\\\.\\A:", 0, 1, &b ) ;

    printf ( "Boot Sector name: %s\n", b.id ) ;
    printf ( "Bytes per Sector: %d\n", b.bps )  ;
    printf ( "Sectors per Cluster: %d\n", b.spc ) ;
    /* rest of the statements can be written by referring Figure 19.6
       and Appendix G*/
}

void ReadSector ( char *src, int ss, int num, void* buff )
{
    HANDLE h ;
    unsigned int br ;
    h = CreateFile  ( src, GENERIC_READ,
                      FILE_SHARE_READ, 0, OPEN_EXISTING, 0, 0 ) ;
    SetFilePointer ( h, ( ss * 512 ), NULL, FILE_BEGIN ) ;
    ReadFile ( h, buff, 512 * num, &br, NULL )  )
    CloseHandle ( h ) ;
```

}

The boot sector contains two parts—'Boot Parameters' and 'Disk Bootstrap Program'. The Boot Parameters are useful while performing read/write operations on the disk. Figure 19.6 shows the break up of the boot parameters for a floppy disk.

Description	**Length**	**Typical Values**
Jump instruction	3	EB3C90
OEM name	8	MSWIN4.1
Bytes per sector	2	512
Sectors per cluster	1	1
Reserved sectors	2	1
Number of FAT copies	1	2
Max. Root directory entries	2	224
Total sectors	2	2880
Media descriptor	1	F0
Sectors per FAT	2	9
Sectors per track	2	18
No. Of sides	2	2
Hidden sectors	4	0
Huge sectors	4	0
BIOS drive number	1	0
Reserved sectors	1	0
Boot signature	1	41
Volume ID	4	349778522
Volume label	11	ICIT
File system type	8	FAT12

Figure 19.6

Using the breakup of bytes shown in Figure 19.6 our program has first created a structure called **boot**. Notice the usage of **#pragma pack** to ensure that all elements of the structure are aligned on a 1-byte boundary, rather than the default 4-byte boundary. Then comes the surprise—there is no **WinMain( )** in the program. This is because we want to display the boot sector contents on the screen rather than in a window. This has been done only for the sake of simplicity. Remember that our aim is to interact with the floppy, and not in drawing and painting in a window. If you wish you can of course adapt this program to display the same contents in a window. So the program is still a Windows application. Only difference is that it is built as a 'Win32 Console Application' using VC++. A console application always begins with **main( )** rather than **WinMain( )**.

To actually read the contents of boot sector of the floppy disk the program makes a call to a user-defined function called **ReadSector( )**. The **ReadSector( )** function is quite similar to the **absread( )** library function available in Turbo C/C++ under DOS.

The first parameter passed to **ReadSector( )** is a string that indicates the storage device from where the reading has to take place. The syntax for this string is **\\machine-name\storage-device name**. In **\\.\A:**, we have used '.' for the machine name. A '.' means the same machine on which the program is executing. Needless to say, **A:** refers to the floppy drive. The second parameter is the logical sector number. We have specified this as 0 which means the boot sector in case of a floppy disk. The third parameter is the number of sectors that we wish to read. This parameter is specified as 1 since the boot sector occupies only a single sector. The last parameter is the address of a buffer/variable that would collect the data that is read from the floppy. Here we have passed the address of the boot structure variable **b**. As a result, the structure variable would be setup with the contents of the boot sector data at the end of the function call.

Once the contents of the boot sector have been read into the structure variable **b** we have displayed the first few of them on the screen using **printf( )**. If you wish you can print the rest of the contents as well.

The *ReadSector( )* Function

With the preliminaries over let us now concentrate on the real stuff in this program, i.e. the **ReadSector( )** function. This function begins by making a call to the **CreateFile( )** API function as shown below:

```
h = CreateFile  ( src, GENERIC_READ,
                  FILE_SHARE_READ, 0, OPEN_EXISTING, 0, 0 ) ;
```

The **CreateFile( )** API function is very versatile. Anytime we are to communicate with a device we have to firstly call this API function. The **CreateFile( )** function opens the specified device as a file. Windows treats all devices just like files on disk. Reading from this file means reading from the device.

The **CreateFile( )** API function takes several parameters. The first parameter is the string specifying the device to be opened. The second parameter is a set of flags that are used to specify the desired access to the file (representing the device) about to be opened. By specifying the **GENERIC_READ** flag we have indicated that we just wish to read from the file (device). The third parameter specifies the sharing access for the file (device). Since floppy drive is a shared resource across all the running applications we have specified the **FILE_SHARE_READ** flag. In general while interacting with any hardware the sharing flag for the file (device) must always be set to this value since every piece of hardware is shared amongst all the running applications. The fourth parameter indicates security access for the file (device). Since we are not concerned with security here we have specified the value as **0**. The fifth parameter specifies what action to take if

the file already exists. When using **CreateFile( )** for device access we must always specify this parameter as **OPEN_EXISTING**. Since the floppy disk file was already opened by the OS a long time back during the booting. The remaining two parameters are not used when using **CreateFile( )** API function for device access. Hence we have passed a **0** value for them. If the call to **CreateFile( )** succeeds then we obtain a handle to the file (device).

The device file mechanism allows us to read from the file (device) by setting the file pointer using the **SetFilePointer( )** API function and then reading the file using the **ReadFile( )** API function. Since every sector is **512** bytes long, to read from the n^{th} sector we need to set the file pointer to the **512 * n** bytes from the start of the file. The first parameter to **SetFilePointer( )** is the handle of the device file that we obtained by calling the **CreateFile( )** function. The second parameter is the byte offset from where the reading is to begin. This second parameter is relative to the third parameter. We have specified the third parameter as **FILE_BEGIN** which means the byte offset is relative to the start of the file.

To actually read from the device file we have made a call to the **ReadFile( )** API function. The **ReadFile( )** function is very easy to use. The first parameter is the handle of the file (device), the second parameter is the address of a buffer where the read contents should be dumped. The third parameter is the count of bytes that have to be read. We have specified the value as **512 * num** so as to read **num** sectors. The fourth parameter to **ReadFile( )** is the address of an **unsigned int** variable which is set up with the count of bytes that the function was successfully able to read. Lastly, once our work with the device is over we should close the file (device) using the **CloseHandle( )** API function.

Though **ReadSector( )** doesn't need it, there does exist a counterpart of the **ReadFile( )** function. Its name is **WriteFile( )**. This API function can be used to write to the file (device). The parameters of **WriteFile( )** are same as that of **ReadFile( )**. Note

that when **WriteFile( )** is to be used we need to specify the **GENERIC_WRITE** flag in the call to **CreateFile( )** API function. Given below is the code of **WriteSector( )** function that works exactly opposite to the **ReadSector( )** function.

```
void WriteSector ( char *src, int ss, int num, void* buff )
{
    HANDLE h ;
    unsigned int br ;
    h = CreateFile  ( src, GENERIC_WRITE,
                      FILE_SHARE_WRITE, 0, OPEN_EXISTING, 0, 0 ) ;
    SetFilePointer ( h, ( ss * 512 ), NULL, FILE_BEGIN ) ;
    WriteFile ( h, buff, 512 * num, &br, NULL )  )
    CloseHandle ( h ) ;
}
```

Accessing Other Storage Devices

Note that the mechanism of reading from or writing to any device remains standard under Windows. We simply need to change the string that specifies the device. Here are some sample calls for reading/writing from/to various devices:

```
ReadSector ( "\\\\.\\a:", 0, 1, &b ) ;  /* reading from 2nd floppy drive */
ReadSector ( "\\\\.\\d:", 0, 1, buffer ) ;  /* reading from a CD-ROM drive */
WriteSector ( "\\\\.\\c:", 0, 1, &b ) ;  /* writing to a hard disk */
ReadSector ( "\\\\.\\physicaldrive0", 0, 1, &b ) ;  /* reading partition table */
```

Here are a few interesting points that you must note.

(a) If we are to read from the second floppy drive we should replace **A:** with **B:** while calling **ReadSector( )**.

(b) To read from storage devices like hard disk drive or CD-ROM or ZIP drive, etc. use the string with appropriate drive letter. The string can be in the range **\\.\C:** to **\\.\Z:**.

(c) To read from the CD-ROM just specify the drive letter of the drive. Note that CD-ROMs follow a different storage organization known as CD File System (CDFS).

(d) The hard disk is often divided into multiple partitions. Details like the place at which each partition begins and ends, the size of each partition, whether it is a bootable partition or not, etc. are stored in a table on the disk. This table is often called 'Partition Table'. If we are to read the partition table contents we can do so by using the string **\\.\physicaldrive0**.

(e) Using **\\.\physicaldrive0** we can also read contents of any other parts of the disk. Here **0** represents the first hard disk in the system. If we are to read from the second hard disk we need to use **1** in place of **0**.

Communication with Keyboard

Like mouse messages there also exist messages for keyboard. These are **WM_KEYDOWN**, **WM_KEYUP** and **WM_CHAR**. Of these, **WM_KEYDOWN** and **WM_KEYUP** are sent to an application (which has the input focus) whenever the key is pressed and released respectively. The additional information in case of these messages is the code of the key being pressed or released. When we tackle **WM_KEYDOWN** or **WM_KEYUP** we need to ourselves check the status of toggle keys like NumLock and CapsLock and shift keys like Ctrl, Alt and Shift. If we wish to avoid all this checking we can tackle the **WM_CHAR** message instead.

What is mentioned above is the normal procedure followed by most Windows applications. However, if we wish to go a step further and deal with the keyboard we need to tackle it differently. For example, suppose we are to perform one of the following jobs:

(a) Once you hit any key CapsLock should become on. Once it becomes on it should remain permanently on.

(b) If we hit a key once it should appear twice on the screen.
(c) If we hit a key A then B should appear on the screen, if we hit a B then C should occur and so on.

Note that all these effects should work on a system-wide basis for all Win32 applications. To be able to achieve these effect we need understand two important mechanisms—'Dynamic Linking' and 'Windows Hooks'. Let us understand these mechanisms one by one.

Dynamic Linking

As we saw in Chapter 16, Windows permits linking of libraries stored in a .DLL file during execution. A .DLL file is a binary file that cannot execute on its own. It contains functions that can be shared between several applications running in memory.

Windows Hooks

As the name suggests, the hook mechanism permits us to intercept and alter the flow of messages in the OS before they reach the application. Since hooks are used to alter the messaging mechanism on a system-wide basis the code for hooking has to be written in a DLL. The hooking mechanism involves writing a hook procedure in a DLL file and registering this procedure with the OS. Since the DLL cannot execute on its own we need a separate program that would load and execute the DLL.

For different messages there are different types of hooks. For example, for keyboard messages there is a keyboard hook, for mouse messages there is mouse hook, etc. You can refer MSDN for nearly a dozen more types of hooks. Here we would restrict our discussion only to the keyboard hook.

Before we proceed to write our own hook procedure let us understand the normal working of the keyboard messages. This is illustrated in Figure 19.7.

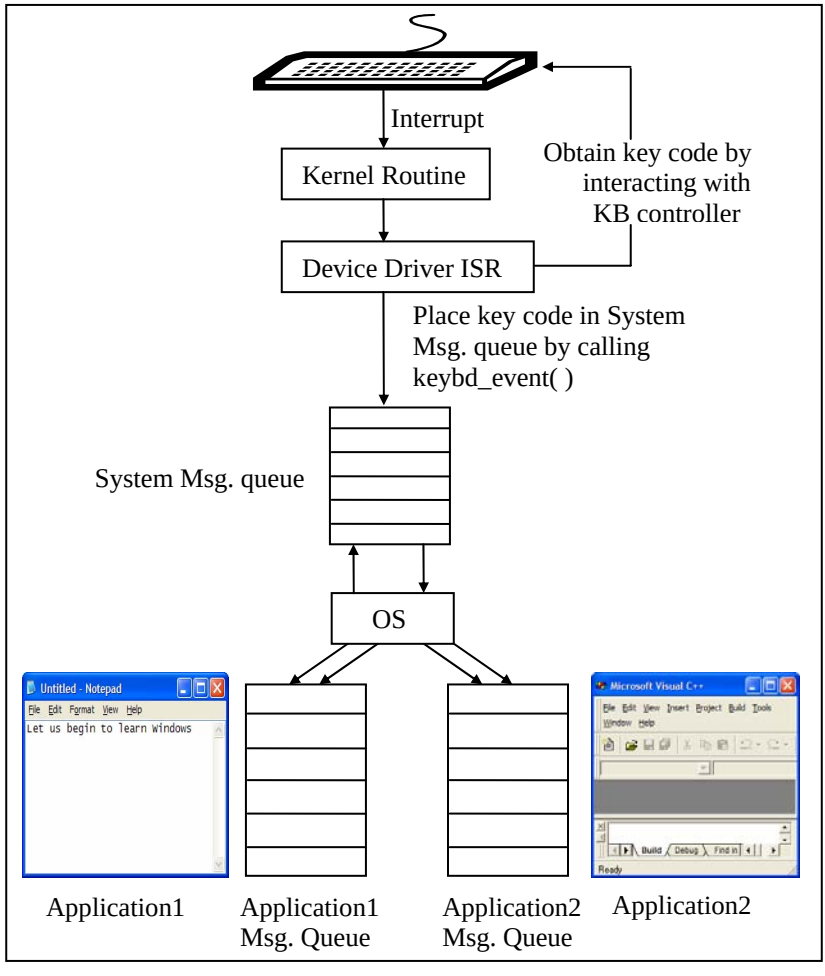

Figure 19.7

With reference to Figure 19.7 here is a list of steps that are carried out when we press a key from the keyboard:

(a) On pressing a key an interrupt occurs and the corresponding kernel routine gets called.
(b) The kernel routine calls the ISR of the keyboard device driver.
(c) The ISR communicates with the keyboard controller and obtains the code of the key pressed.
(d) The ISR calls a OS function **keybd_event( )** to post the key code to the System Message Queue.
(e) The OS retrieves the message from the System Message Queue and posts it into the message queue of the application with regard to which the key has been pressed.

Let us now see what needs to be done if we are to alter this procedure. We simply need to register our hook procedure with the OS. As a result, our hook procedure would receive the message before it is dispatched to the appropriate Application Message Queue. Since our hook procedure gets a first shot at the message it can now alter the working in the following three ways:

(a) It can suppress the message altogether
(b) It can change the message
(c) It can post more messages into the System Message Queue using the **keybd_event( )** function.

Let us now put all this theory into practice by writing a few programs.

Caps Locked, Permanently

Let us now write a program that keeps the CapsLock permanently on. This effect would come into being when the first key is hit subsequent to the execution of our program. In fact there would be two programs:

(a) A DLL containing a hook procedure that achieves the CapsLock effect.
(b) An application EXE which loads the DLL in memory.

Given below is the source code of the DLL program.

```
/* hook.c */

# include <windows.h>

static HHOOK hkb = NULL ;
HANDLE h ;

BOOL __stdcall DllMain ( HANDLE hModule, DWORD ul_reason_for_call,
                         LPVOID lpReserved )
{
    h = hModule ;
    return TRUE ;
}

BOOL __declspec ( dllexport ) installhook( )
{
    hkb = SetWindowsHookEx ( WH_KEYBOARD,
                    ( HOOKPROC ) KeyboardProc, ( HINSTANCE ) h, 0 ) ;
    if ( hkb == NULL )
        return FALSE ;

    return TRUE ;
}

LRESULT __declspec ( dllexport ) __stdcall KeyboardProc ( int nCode,
                                  WPARAM wParam, LPARAM lParam )
{
    short int state ;

    if ( nCode < 0 )
        return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;

    if ( ( nCode == HC_ACTION ) &&
                    ( ( DWORD ) lParam & 0x40000000 ) )
    {
        state = GetKeyState ( VK_CAPITAL ) ;
        if ( (state & 1 )== 0)  /* if off */
```

```
        {
            keybd_event ( VK_CAPITAL , 0,
                          KEYEVENTF_EXTENDEDKEY, 0 ) ;
            keybd_event ( VK_CAPITAL , 0,
            KEYEVENTF_EXTENDEDKEY | KEYEVENTF_KEYUP, 0 ) ;
        }
    }
    return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;
}

BOOL __declspec ( dllexport )  removehook( )
{
    return UnhookWindowsHookEx ( hkb ) ;
}
```

Follow the steps mentioned below to create this program:

(a) Select 'File | New' option to start a new project in VC++.
(b) From the 'Project' tab select 'Win32 Dynamic-Link Library' and click on the 'Next' button.
(c) In the 'Win32 Dynamic-link Library Step 1 of 1' select "An empty DLL project" and click on the 'Finish' button.
(d) Select 'File | New' option.
(e) From the 'File' tab select 'C++ source file' and give the file name as 'hook.c'. Type the code listed above in this file.
(f) Compile the program to generate the .DLL file.

Note that this program doesn't contain **WinMain( )** since the program on compilation should not execute on its own. It has been replaced by a function called **DllMain( )**. This function acts as entry point of the DLL program. It gets called when the DLL is loaded or unloaded.

When the application loads the DLL the **DllMain( )** function would be called. In this function we have merely stored the handle to the DLL that has been loaded in memory into a global variable **h** for later use.

Those functions in a DLL that can be called from outside it are called exported functions. Our DLL contains three such functions—**installhook( )**, **removehook( )** and **KeyboardProc( )**. To indicate to the compiler that a function in a DLL is an exported function we have to pre-qualify it with **__declspec (dllexport)**. These functions would be called from the second program. This second program is a normal GUI application created in the same way that we did applications in Chapters 17 and 18. The handlers for messages **WM_CREATE** and **WM_DESTROY** are given below:

```
/* capslocked.c */

HINSTANCE h ;

void OnCreate ( HWND hWnd )
{
    BOOL ( CALLBACK *p )( ) ;

    h = LoadLibrary ( "hook.dll" ) ;
    if ( h != NULL )
    {
        p = GetProcAddress ( h, "installhook" ) ;
        ( *p )( ) ;  /* calls installhoook( ) function */
    }
}

void OnDestroy ( HWND hWnd )
{
    BOOL ( CALLBACK *p )( ) ;

    p = GetProcAddress ( h, "removehook" ) ;
    ( *p )( ) ;  /* calls removehoook( ) function */

    FreeLibrary ( h ) ;
    PostQuitMessage ( 0 ) ;
}
```

As we know, the **OnCreate( )** and **OnDestroy( )** handlers would be called when the **WM_CREATE** and **WM_DESTROY** messages arrive respectively. In **OnCreate( )** we have loaded the DLL containing the hook procedure. To do this we have called the **LoadLibrary( )** API function. Once the DLL is loaded we have obtained the address of the exported function **installhook( )** using the **GetProcAddress( )** API function. The returned address is stored in **p**, where **p** is a pointer to the **installhook( )** function. Using this pointer we have then called the **installhook( )** function.

In the **installhook( )** function we have called the API function **SetWindowsHookEx( )** to register our hook procedure with the OS as shown below:

```
hkb = SetWindowsHookEx ( WH_KEYBOARD,
                    ( HOOKPROC ) KeyboardProc, ( HINSTANCE ) h, 0 ) ;
```

Here the first parameter is the type of hook that we wish to register, whereas the second parameter is the address of our hook procedure **KeyboardProc( )**. **hkb** stores the handle of the hook installed.

From now on whenever a keyboard message is retrieved by the OS from the System Message Queue the message is firstly passed to our hook procedure, i.e. to **KeyboardProc( )** function. Inside this function we have written code to ensure that the CapsLock always remains on. To begin with we have checked whether **nCode** parameter is less than **0**. If it so then it necessary to call the next hook procedure. The MSDN documentation suggests that "if code is less than zero, the hook procedure must pass the message to the **CallNextHookEx( )** function without further processing and should return the value returned by **CallNextHookEx( )**".

Note that there can be several hook procedures installed by different programs, thus forming a chain of hook procedures. These hook procedures always get called in an order that is

opposite to their order of installation. This means the last hook procedure installed is the first one to get called.

If the **nCode** parameter contains a value **HC_ACTION** it means that the message that was just removed form the system message queue was a keyboard message. If it is so, then we have checked the previous state of the key before the message was sent. If the state of the key was 'depressed' (30^{th} bit of **lParam** is 1) then we have obtained the state of the CapsLock key by calling the **GetKeyState( )** API function. If it is off (0^{th} bit of **state** variable is 0) then we have turned on the CapsLock by simulating a keypress. For this simulation we have called the function **keybd_event( )** twice—first call is for pressing the CapsLock and second is for releasing it. Note that **keybd_event( )** creates a keyboard message from the parameters that we pass to it and posts it into the system message queue. The parameter VK_CAPITAL represents the code for the CapsLock key.

A word of caution! When we use **keybd_event( )** to post keyboard message for a simulated CapsLock keypress, once again our hook procedure would be called when these messages are retrieved from the system message queue. But this time the CapsLock would be on so we would end up passing control to the next hook procedure through a call to **CallNextHookEx( )**.

When we close the application window as usual the **OnDestroy( )** would be called. In this handler we have obtained the address of the **removehook( )** exported function and called it. In the **removehook( )** function we have unregistered our hook procedure by calling the **UnhookWindowsHookEx( )** API function. Note that to this function we have passed the handle to our hook. As a result our hook procedure is now removed from the hook chain. Hereafter the CapsLock would behave normally. Having unhooked our hook procedure the control would return to **OnDestroy( )** handler where we have promptly unload the DLL from memory by calling the **FreeLibrary( )** API function.

One last point about this program—the 'hook.dll' file should be copied into the directory of the application's EXE before executing the EXE.

Did You Press It TTwwiiccee….

With the power of windows hooks below your belt you are into the league of power programmers of Windows. So how about tasting the power some bit more. How about writing a program that would make every key pressed in any Windows application appear twice. Here is the code for the hook procedure.

```
LRESULT __declspec ( dllexport ) __stdcall KeyboardProc ( int nCode,
                                        WPARAM wParam, LPARAM lParam )
{
    static BYTE key  ;
    static BOOL flag = FALSE ;

    if ( nCode < 0 )
        return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;

    if ( ( nCode == HC_ACTION ) &&
        ( ( DWORD ) lParam & 0x80000000 ) == 0 )
    {
        if ( flag == FALSE )
        {
            key = wParam ;
            keybd_event ( key , 0, KEYEVENTF_EXTENDEDKEY, 0 ) ;
            flag = TRUE ;
        }
        else
        {
            if ( key == ( BYTE ) wParam )
                flag = FALSE ;
        }
    }
    return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;
```

```
}
```

In this hook procedure once again we have checked if the **nCode** parameter contains a value **HC_ACTION**. If it does then we have checked the present state of the key in question. If the present state of the key is 'pressed' (31^{th} bit of **lParam** is 0) then we have posted the message for the same key into the system message queue by calling the **keybd_event( )**. However, this may lead to a serious problem. Can you imagine which? The message that we post, once retrieved, would again bring the control to our hook procedure. Once again the conditions would become true and we would post the same message again. This would go on and on. This can be prevented by using a using a simple **flag** variable as shown in the code.

Note that the rest of the functions in the DLL file are exactly same as in the previous program. So also is the application program.

Mangling Keys

How about one more program to bolster your confidence? Let us try one that would mangle every key that is pressed. That is, convert an A to a B, B to C, C to D, etc. This would be fairly straight-forward. We simply have to increment the key code before posting it into the system message queue. Also, further processing of key has to be prevented. This can be achieved by simply returning a non-zero value from the hook procedure (thus bypassing the call to **CallNextHookEx( )**). This is shown in the following hook procedure.

```
LRESULT __declspec ( dllexport ) __stdcall KeyboardProc ( int nCode,
                                        WPARAM wParam, LPARAM lParam )
{
    static BYTE key ;
    static BOOL flag = FALSE ;
```

```
    if ( nCode < 0 )
        return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;

    if ( ( nCode == HC_ACTION ) &&
       ( ( DWORD ) lParam & 0x80000000 ) == 0 )
    {
        if ( flag == FALSE )
        {
            key = wParam ;
            key ++ ;
            keybd_event ( key , 0, KEYEVENTF_EXTENDEDKEY, 0 ) ;
            flag = TRUE ;
            return 1 ;
        }
        else
        {
            if ( key == ( BYTE ) wParam )
                flag = FALSE ;
        }
    }
    return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;
}
```

KeyLogger

There are several malicious programs that are floating on the net that steal away your passwords. These programs keep a log of every key that is pressed while entering passwords or credit card numbers. These programs make use of windows hooks to trap every key that is pressed. With the knowledge that you have gained from the past three programs this may not be a big deal.

However, such key logger programs deviate from the ones that we developed in three fundamental ways:

(a) They do not pop any window on the screen; otherwise the program's presence would get detected.

(b) These programs also hide themselves from the Task Manager so that the user cannot terminate them.

(c) The logged keys are secretly sent over the net to the malicious users who write such programs. Once the logged keys are known it would be possible to break into the system.

Where is This Leading

Even for a moment do not create an impression in you mind that Windows Hooks are only for notorious activities. There are many good things that they can be put to use for. These activities include:

(a) Multimedia keyboards have special key like Cut, Copy, Paste, etc. Such keyboards also come with special programs which when installed know how to tackle these special keys. On pressing these keys these programs use the hook mechanism to place the simulated keys in the system message queue.

(b) Many demo programs once executed automatically move the mouse pointer to a menu or a toolbar or any such item to demonstrate some feature of the software. To manage these actions a windows hook called Journal hook is used.

(c) For physically impaired persons a keyboard can be simulated on the screen and the mouse clicks on this keyboard can be communicated to Windows as actual key hits. This again can be achieved using mouse and keyboard hook.

There can be many more such examples. But the above three I believe would be ample to prove to you the constructive side of the powerful mechanism called Windows Hooks.

Summary

(a) Hardware interaction can happen in two ways: (1) When the user interacts with the hardware and the program reacts to it. (2) When the program interacts with the hardware without any user intervention.

(b) In DOS when the user interacts with the hardware an ISR gets called which interacts with the hardware. In Windows the same thing is done by the device driver's ISR.

(c) In DOS when the program has to interact with the hardware it can do so by using library functions, DOS/BIOS routines or by directly interacting with the hardware. In Windows the same thing can be done by using API functions.

(d) Under Windows to gain finer control over the hardware we are required to write a device driver program.

(e) Interaction with the any device can be done using API functions like **CreateFile( )**, **ReadFile( )**, **WriteFile( )** and **CloseHandle( )**.

(f) Different strings have to be passed to the **CreateFile( )** functions for interacting with different devices.

(g) Windows provides a powerful mechanism called hooks that can alter the flow of messages before they reach the application.

(h) Windows hook procedures should be written in a DLL since they work on a system wide basis.

(i) Windows hooks can be put to many good uses.

Exercise

[A] State True or False:

(a) In MS-DOS on occurrence of an interrupt values from IDT are used to call the appropriate kernel routine.

(b) Under Windows on occurrence of an interrupt the kernel routine calls the appropriate device driver's ISR.

(c) Under Windows an application can interact with the hardware by directly calling its device driver's routines.

(d) Under Windows we can write device drivers to extend the OS itself.
(e) **ReadSector( )** and **WriteSector( )** are API functions.
(f) While reading a sector from the disk the **CreateFile( )** function creates a file on the disk.
(g) The Windows API function to stop communication with a device is **CloseFile( )**.
(h) The **ReadFile( )** and **WriteFile( )** API functions can only perform reading or writing from/to a disk file.

[B] Answer the following:

(a) How is hardware interaction under Windows different that that under DOS?

(b) What is the advantage of writing code in a DLL?

(c) Explain the Windows hooks mechanism.

(d) What is the standard way of communicating with a device under Windows?

(e) Write a program to read the contents of Boot Sector of a 32-bit FAT file system and print them on the screen. Refer Appendix G for details about the contents of the boot sector.

(f) Write a program that ensures that the key 'A' is completely disabled across all applications.

(g) Write a program that closes any window just by placing the cursor on the 'Close' button in the title bar of it.

20 C Under Linux

- What is Linux
- C Programming Under Linux
- The ‘Hello Linux’ Program
- Processes
- Parent and Child Processes
- More Processes
- Zombies and Orphans
- One Interesting Fact
- Summary
- Exercise

Today the programming world is divided into two major camps—the Windows world and the Linux world. Since its humble beginning about a decade ago, Linux has steadily drawn the attention of programmers across the globe and has successfully created a community of its own. How big and committed is this community is one of the hottest debates that is raging in all parts of the world. You can look at the hot discussions and the flame wars on this issue on numerous sites on the internet. Before you decide to join the Windows or the Linux camp you should first get familiar with both of them. The last 4 chapters concentrated on Windows programming. This and the next one would deal with Linux programming. Without any further discussions let us now set out on the Linux voyage. I hope you find the journey interesting and exciting.

What is Linux

Linux is a clone of the Unix operating system. Its kernel was written from scratch by Linus Torvalds with assistance from a loosely-knit team of programmers across the world on Internet. It has all the features you would expect in a modern OS. Moreover, unlike Windows or Unix, Linux is available completely free of cost. The kernel of Linux is available in source code form. Anybody is free to change it to suit his requirement, with a precondition that the changed kernel can be distributed only in the source code form. Several programs, frameworks, utilities have been built around the Linux kernel. A common user may not want the headaches of downloading the kernel, going through the complicated compilation process, then downloading the frameworks, programs and utilities. Hence many organizations have come forward to make this job easy. They distribute the precompiled kernel, programs, utilities and frameworks on a common media. Moreover, they also provide installation scripts for easy installations of the Linux OS and applications. Some of the popular distributions are RedHat, SUSE, Caldera, Debian, Mandrake, Slackware, etc. Each of them contain the same kernel

but may contain different application programs, libraries, frameworks, installation scripts, utilities, etc. Which one is better than the other is only a matter of taste.

Linux was first developed for x86-based PCs (386 or higher). These days it also runs on Compaq Alpha AXP, Sun SPARC, Motorola 68000 machines (like Atari ST and Amiga), MIPS, PowerPC, ARM, Intel Itanium, SuperH, etc. Thus Linux works on literally every conceivable microprocessor architecture.

Under Linux one is faced with simply too many choices of Linux distributions, graphical shells and managers, editors, compilers, linkers, debuggers, etc. For simplicity (in my opinion) I have chosen the following combination:

Linux Distribution - Red Hat Linux 9.0
Console Shell - BASH
Graphical Shell - KDE 3.1-10
Editor - KWrite
Compiler - GNU C and C++ compiler (gcc)

We would be using and discussing these in the sections to follow.

C Programming Under Linux

How is C under Linux any different than C under DOS or C under Windows? Well, it is same as well as different. It is same to the extent of using language elements like data types, control instructions and the overall syntax. The usage of standard library functions is also same even though the implementation of each might be different under different OS. For example, a **printf( )** would work under all OSs, but the way it is defined is likely to be different for different OSs. The programmer however doesn't suffer because of this since he can continue to call **printf( )** the same way no matter how it is implemented.

But there the similarity ends. If we are to build programs that utilize the features offered by the OS then things are bound to be different across OSs. For example, if we are to write a C program that would create a Window and display a message "hello" at the point where the user clicks the left mouse button. The architecture of this program would be very closely tied with the OS under which it is being built. This is because the mechanisms for creating a window, reporting a mouse click, handling a mouse click, displaying the message, closing the window, etc. are very closely tied with the OS for which the program is being built. In short the programming architecture (better known as programming model) for each OS is different. Hence naturally the program that achieves the same task under different OS would have to be different.

The 'Hello Linux' Program

As with any new platform we would begin our journey in the Linux world by creating a 'hello world' program. Here is the source code....

```
int main( )
{
    printf ( "Hello Linux\n" ) ;
    return 0 ;
}
```

The program is exactly same as compared to a console program under DOS/Windows. It begins with **main( )** and uses **printf( )** standard library function to produce its output. So what is the difference? The difference is in the way programs are typed, compiled and executed. The steps for typing, compiling and executing the program are discussed below.

The first hurdle to cross is the typing of this program. Though any editor can be used to do so, we have preferred to use the editor called 'KWrite'. This is because it is a very simple yet elegant

editor compared to other editors like 'vi' or 'emacs'. Note that KWrite is a text editor and is a part of K Desktop environment (KDE). Installation of Linux and KDE is discussed in Appendix H. Once KDE is started select the following command from the desktop panel to start KWrite:

K Menu | Accessories | More Accessories | KWrite

If you face any difficulty in starting the KWrite editor please refer Appendix H. Assuming that you have been able to start KWrite successfully, carry out the following steps:

(a) Type the program and save it under the name 'hello.c'.
(b) At the command prompt switch to the directory containing 'hello.c' using the **cd** command.
(c) Now compile the program using the **gcc** compiler as shown below:

```
# gcc hello.c
```

(d) On successful compilation **gcc** produces a file named 'a.out'. This file contains the machine code of the program which can now be executed.
(e) Execute the program using the following command.

```
# ./a.out
```

(f) Now you should be able to see the output 'Hello Linux' on the screen.

Having created a Hello Linux program and gone through the edit-compile-execute cycle once let us now turn our attention to Linux specific programming. We will begin with processes.

Processes

Gone are the days when only one job (task) could be executed in memory at any time. Today the modern OSs like Windows and

Linux permit execution of several tasks simultaneously. Hence these OSs are aptly called 'Multitasking' OSs.

In Linux each running task is known as a 'process'. Even though it may appear that several processes are being executed by the microprocessor simultaneously, in actuality it is not so. What happens is that the microprocessor divides the execution time equally among all the running processes. Thus each process gets the microprocessor's attention in a round robin manner. Once the time-slice allocated for a process expires the operation that it is currently executing is put on hold and the microprocessor now directs its attention to the next process. Thus at any given moment if we take the snapshot of memory only one process is being executed by the microprocessor. The switching of processes happens so fast that we get a false impression that the processor is executing several processes simultaneously.

The scheduling of processes is done by a program called 'Scheduler' which is a vital component of the Linux OS. This scheduler program is fairly complex. Before switching over to the next thread it stores the information about the current process. This includes current values of CPU registers, contents of System Stack and Application Stack, etc. When this process again gets the time slot these values are restored. This process of shifting over from one thread to another is often called a Context Switch. Note that Linux uses preemptive scheduling, meaning thereby that the context switch is performed as soon as the time slot allocated to the process is over, no matter whether the process has completed its job or not.

Kernel assigns each process running in memory a unique ID to distinguish it from other running processes. This ID is often known as processes ID or simply PID. It is very simple to print the PID of a running process programmatically. Here is the program that achieves this…

```
int main( )
{
    printf ( "Process ID = %d", getpid( ) ) ;
}
```

Here **getpid( )** is a library function which returns the process ID of the calling process. When the execution of the program comes to an end the process stands terminated. Every time we run the program a new process is created. Hence the kernel assigns a new ID to the process each time. This can be verified by executing the program several times—each time it would produce a different output.

Parent and Child Processes

As we know, our running program is a process. From this process we can create another process. There is a parent-child relationship between the two processes. The way to achieve this is by using a library function called **fork( )**. This function splits the running process into two processes, the existing one is known as parent and the new process is known as child. Here is a program that demonstrates this…

```
# include <sys/types.h>
int main( )
{
    printf ( "Before Forking\n" ) ;
    fork( ) ;
    printf ( "After Forking\n" ) ;
}
```

Here is the output of the program…

```
Before Forking
After Forking
After Forking
```

Watch the output of the program. You can notice that all the statements after the **fork( )** are executed twice—once by the parent process and second time by the child process. In other words **fork( )** has managed to split our process into two.

But why on earth would we like to do this? At times we want our program to perform two jobs simultaneously. Since these jobs may be inter-related we may not want to create two different programs to perform them. Let me give you an example. Suppose we want perform two jobs—copy contents of source file to target file and display an animated GIF file indicating that the file copy is in progress. The GIF file should continue to play till file copy is taking place. Once the copying is over the playing of the GIF file should be stopped. Since both these jobs are inter-related they cannot be performed in two different programs. Also, they cannot be performed one after another. Both jobs should be performed simultaneously.

At such times we would want to use **fork( )** to create a child process and then write the program in such a manner that file copy is done by the parent and displaying of animated GIF file is done by the child process. The following program shows how this can be achieved. Note that the issue here is to show how to perform two different but inter-related jobs simultaneously. Hence I have skipped the actual code for file copying and playing the animated GIF file.

```
# include <sys/types.h>

int main( )
{
    int  pid ;
    pid = fork( ) ;
    if ( pid == 0 )
    {
        printf ( "In child process\n" ) ;
        /* code to play animated GIF file */
```

```
    }
    else
    {
        printf ( "In parent process\n" ) ;
        /* code to copy file */
    }
}
```

As we know, **fork( )** creates a child process and duplicates the code of the parent process in the child process. There onwards the execution of the **fork( )** function continues in both the processes. Thus the duplication code inside **fork( )** is executed once, whereas the remaining code inside it is executed in both the parent as well as the child process. Hence control would come back from **fork( )** twice, even though it is actually called only once. When control returns from **fork( )** of the parent process it returns the PID of the child process, whereas when control returns from **fork( )** of the child process it always returns a 0. This can be exploited by our program to segregate the code that we want to execute in the parent process from the code that we want to execute in the child process. We have done this in our program using an **if** statement. In the parent process the 'else block' would get executed, whereas in the child process the 'if block' would get executed.

Let us now write one more program. This program would use the **fork( )** call to create a child process. In the child process we would print the PID of child and its parent, whereas in the parent process we would print the PID of the parent and its child. Here is the program…

```
# include <sys/types.h>
int main( )
{
    int pid ;
    pid = fork( ) ;

    if ( pid == 0 )
```

```
    {
        printf ( "Child : Hello I am the child process\n" ) ;
        printf ( "Child : Child's PID: %d\n", getpid( ) ) ;
        printf ( "Child : Parent's PID: %d\n", getppid( ) ) ;
    }
    else
    {
        printf ( "Parent : Hello I am the parent process\n" ) ;
        printf ( "Parent : Parent's PID: %d\n", getpid( ) ) ;
        printf ( "Parent : Child's PID: %d\n", pid ) ;
    }
}
```

Given below is the output of the program:

```
Child : Hello I am the child process
Child : Child's PID: 4706
Child : Parent's PID: 4705
Parent : Hello I am the Parent process
Parent : Parent's PID: 4705
Parent : Child's PID: 4706
```

In addition to **getpid( )** there is another related function that we have used in this program—**getppid( )**. As the name suggests, this function returns the PID of the parent of the calling process.

You can tally the PIDs from the output and convince yourself that you have understood the **fork( )** function well. A lot of things that follow use the **fork( )** function. So make sure that you understand it thoroughly.

Note that even Linux internally uses **fork( )** to create new child processes. Thus there is a inverted tree like structure of all the processes running in memory. The father of all these processes is a process called **init**. If we want to get a list of all the running processes in memory we can do so using the **ps** command as shown below.

```
# ps –A
```

Here the switch **–A** indicates that we want to list all the running processes.

More Processes

Suppose we want to execute a program on the disk as part of a child process. For this first we should create a child process using **fork( )** and then from within the child process we should call an **exec** function to execute the program on the disk as part of a child process. Note that there is a family of **exec** library functions, each basically does the same job but with a minor variation. For example, **execl( )** function permits us to pass a list of command line arguments to the program to be executed. **execv( )** also does the same job as **execl( )** except that the command line arguments can be passed to it in the form of an array of pointers to strings. There also exist other variations like **execle( )** and **execvp( )**.

Let us now see a program that uses **execl( )** to run a new program in the child process.

```
# include <unistd.h>
int main( )
{
      int pid ;
      pid = fork( ) ;
      if ( pid == 0 )
      {
            execl ( "/bin/ls","-al", "/etc", NULL ) ;
            printf ( "Child: After exec( )\n") ;
      }
      else
            printf ( "Parent process\n" ) ;
}
```

After forking a child process we have called the **execl( )** function. This function accepts variable number of arguments. The first parameter to **execl( )** is the absolute path of the program to be executed. The remaining parameters describe the command line arguments for the program to be executed. The last parameter is an end of argument marker which must always be **NULL**. Thus in our case the we have called upon the **execl( )** function to execute the **ls** program as shown below

```
ls -al /etc
```

As a result, all the contents of the **/etc** directory are listed on the screen. Note that the **printf( )** below the call to **execl( )** function is not executed. This is because the **exec** family functions overwrite the image of the calling process with the code and data of the program that is to be executed. In our case the child process's memory was overwritten by the code and data of the **ls** program. Hence the call to **printf( )** did not materialize.

It would make little sense in calling **execl( )** before **fork( )**. This is because a child would not get created and **execl( )** would simply overwrite the main process itself. As a result, no statement beyond the call to **execl( )** would ever get executed. Hence **fork( )** and **execl( )** usually go hand in hand.

Zombies and Orphans

We know that the **ps –A** command lists all the running processes. But from where does the **ps** program get this information? Well, Linux maintains a table containing information about all the processes. This table is called 'Process Table'. Apart from other information the process table contains an entry of 'exit code' of the process. This integer value indicates the reason why the process was terminated. Even though the process comes to an end its entry would remain in the process table until such time that the parent of the terminated process queries the exit code. This act of querying

deletes the entry of the terminated process from the process table and returns the exit code to the parent that raised the query.

When we fork a new child process and the parent and the child continue to execute there are two possibilities—either the child process ends first or the parent process ends first. Let us discuss both these possibilities.

(a) Child terminates earlier than the parent

In this case till the time parent does not query the exit code of the terminated child the entry of the child process would continue to exist. Such a process in Linux terminology is known as a 'Zombie' process. Zombie means ghost, or in plain simple Hindi a 'Bhoot'. Moral is, a parent process should query the process table immediately after the child process has terminated. This would prevent a zombie.

What if the parent terminates without querying. In such a case the zombie child process is treated as an 'Orphan' process. Immediately, the father of all processes—**init**—adopts the orphaned process. Next, as a responsible parent **init** queries the process table as a result of which the child process entry is eliminated from the process table.

(b) Parent terminates earlier than the child

Since every parent process is launched from the Linux shell, the parent of the parent is the **shell** process. When our parent process terminates, the **shell** queries the process table. Thus a proper cleanup happens for the parent process. However, the child process which is still running is left orphaned. Immediately the **init** process would adopt it and when its execution is over **init** would query the process table to clean up the entry for the child process. Note that in this case the child process does not become a zombie.

Thus, when a zombie or an orphan gets created the OS takes over and ensures that a proper cleanup of the relevant process table

entry happens. However, as a good programming practice our program should get the exit code of the terminated process and thereby ensure a proper cleanup. Note that here cleanup is important (it happens anyway). Why is it important to get the exit code of the terminated process. It is because, it is the exit code that would give indication about whether the job assigned to the process was completed successfully or not. The following program shows how this can be done.

```
# include <unistd.h>
# include <sys/types.h>
int main( )
{
    unsigned int i = 0 ;
    int pid, status ;
    pid = fork( ) ;
    if ( pid == 0 )
    {
        while ( i < 4294967295U )
            i++ ;
        printf ( "The child is now terminating\n" ) ;
    }
    else
    {
        waitpid ( pid, &status, 0 )  ;
        if ( WIFEXITED ( status ) )
            printf ( "Parent: Child terminated normally\n" ) ;
        else
            printf ( "Parent: Child terminated abnormally\n" ) ;
    }
    return 0 ;
}
```

In this program we have applied a big loop in the child process. This loop ensures that the child does not terminate immediately. From within the parent process we have made a call to the **waitpid( )** function. This function makes the parent process wait

till the time the execution of the child process does not come to an end. This ensures that the child process never becomes orphaned. Once the child process, terminates the **waitpid( )** function queries its exit code and returns back to the parent. As a result of querying, the child process does not become a zombie.

The first parameter of **waitpid( )** function is the pid of the child process for which the wait has to be performed. The second parameter is the address of an integer variable which is set up with the exit status code of the child process. The third parameter is used to specify some options to control the behavior of the wait operation. We have not used this parameter and hence we have passed a **0**. Next we have made use of the **WIFEXITED( )** macro to test if the child process exited normally or not. This macro takes the status value as a parameter and returns a non-zero value if the process terminated normally. Using this macro the parent suitably prints a message to report the status (normal/abnormal) termination of its child process.

One Interesting Fact

When we use **fork( )** to create a child process the child process does not contain the entire data and code of the parent process. Then does it mean that the child process contains the data and code below the **fork( )** call. Even this is not so. In actuality the code never gets duplicated. Linux internally manages to intelligently share it. As against this, some data is shared, some is not. Till the time both the processes do not change the value of the variables they keep getting shared. However, if any of the processes (either child or parent) attempt to change the value of a variable it is no longer shared. Instead a new copy of the variable is made for the process that is attempting to change it. This not only ensures data integrity but also saves precious memory.

Summary

(a) Linux is a free OS whose kernel was built by Linus Trovalds and friends.
(b) A Linux distribution consists of the kernel with source code along with a large collection of applications, libraries, scripts, etc.
(c) C programs under Linux can be compiled using the popular **gcc** compiler.
(d) Basic scheduling unit in Linux is a 'Process'. Processes are scheduled by a special program called 'Scheduler'.
(e) **fork( )** library function can be used to create child processes.
(f) **Init** process is the father of all processes.
(g) **execl( )** library function is used to execute another program from within a running program,.
(h) **execl( )** function overwrites the image (code and data) of the calling process.
(i) **execl( )** and **fork( )** usually go hand in hand.
(j) **ps** command can be used to get a list of all processes.
(k) **kill** command can be used to terminate a process.
(l) A 'Zombie' is a child process that has terminated but its parent is running and has not called a function to get the exit code of the child process.
(m) An 'Orphan' is a child process whose parent has terminated.
(n) Orphaned processes are adopted by **init** process automatically.
(o) A parent process can avoid creation of a Zombie and Orphan processes using **waitpid( )** function.

Exercise

[A] State True or False:

(a) We can modify the kernel of Linux OS.
(b) All distributions of Linux contain the same collection of applications, libraries and installation scripts.
(c) Basic scheduling unit in Linux is a file.

(d) **execl( )** library function can be used to create a new child process.
(e) The scheduler process is the father of all processes.
(f) A family of **fork( )** and **exec( )** functions are available, each doing basically the same job but with minor variations.
(g) **fork( )** completely duplicates the code and data of the parent process into the child process.
(h) **fork( )** overwrites the image (code and data) of the calling process.
(i) **fork( )** is called twice but returns once.
(j) Every zombie process is essentially an orphan process.
(k) Every orphan process is essentially an orphan process.

[B] Answer the following:

(a) If a program contains four calls to **fork( )** one after the other how many total processes would get created?

(b) What is the difference between a zombie process and an orphan process?

(c) Write a program that prints the command line arguments that it receives. What would be the output of the program if the command line argument is * ?

(d) What purpose do the functions **getpid( )**, **getppid( )**, **getpppid( )** serve?

(e) Rewrite the program in the section 'Zombies and Orphans' replacing the **while** loop with a call to the **sleep( )** function. Do you observe any change in the output of the program?

(f) How does **waitpid( )** prevent creation of Zombie or Orphan processes?

21 More Linux Programming

- Communication using Signals
- Handling Multiple Signals
- Registering a Common Handler
- Blocking Signals
- Event driven programming
- Where Do You Go From Here
- Summary
- Exercise

Communication is the essence of all progress. This is true in real life as well as in programming. In today's world a program that runs in isolation is of little use. A worthwhile program has to communicate with the outside world in general and with the OS in particular. In Chapters 16 and 17 we saw how a Windows based program communicates with Windows. In this chapter let us explore how this communication happens under Linux.

Communication using Signals

In the last chapter we used **fork( )** and **exec( )** library function to create a child process and to execute a new program respectively. These library functions got the job done by communication with the Linux OS. Thus the direction of communication was from the program to the OS. The reverse communication—from the OS to the program—is achieved using a mechanism called 'Signal'. Let us now write a simple program that would help you experience the signal mechanism.

```
int main( )
{
    while ( 1 )
        printf ( "Pogram Running\n" ) ;
    return 0 ;
}
```

The program is fairly straightforward. All that we have done here is we have used an infinite **while** loop to print the message "Program Running" on the screen. When the program is running we can terminate it by pressing the Ctrl + C. When we press Ctrl + C the keyboard device driver informs the Linux kernel about pressing of this special key combination. The kernel reacts to this by sending a signal to our program. Since we have done nothing to handle this signal the default signal handler gets called. In this

default signal handler there is code to terminate the program. Hence on pressing Ctrl + C the program gets terminated.

But how on earth would the default signal handler get called. Well, it is simple. There are several signals that can be sent to a program. A unique number is associated with each signal. To avoid remembering these numbers, they have been defined as macros like **SIGINT**, **SIGKILL**, **SIGCONT**, etc. in the file 'signal.h'. Every process contains several 'signal ID - function pointer' pairs indicating for which signal which function should be called. If we do not decide to handle a signal then against that signal ID the address of the default signal handler function is present. It is precisely this default signal handler for **SIGINT** that got called when we pressed Ctrl + C when the above program was executed. INT in **SIGINT** stands for interrupt.

Let us know see how can we prevent the termination of our program even after hitting Ctrl + C. This is shown in the following program:

```
# include <signal.h>

void sighandler ( int signum )
{
    printf ( "SIGINT received. Inside sighandler\n" ) ;
}

int main( )
{
    signal ( SIGINT, ( void* ) sighandler ) ;
    while ( 1 )
        printf ( "Program Running\n" ) ;
    return 0 ;
}
```

In this program we have registered a signal handler for the SIGINT signal by using the **signal( )** library function. The first parameter

of this function specifies the ID of the signal that we wish to register. The second parameter is the address of a function that should get called whenever the signal is received by our program. This address has to be typecasted to a **void *** before passing it to the **signal( )** function.

Now when we press Ctrl + C the registered handler, namely, **sighandler( )** would get called. This function would display the message 'SIGINT received. Inside sighandler' and return the control back to **main( )**. Note that unlike the default handler, our handler does not terminate the execution of our program. So only way to terminate it is to kill the running process from a different terminal. For this we need to open a new instance of command prompt (terminal). How to start a new instace of command prompt is discussed in Appendix H. Next do a **ps –a** to obtain the list of processes running at all the command prompts that we have launched. Note down the process id of **a.out**. Finally kill 'a.out' process by saying

```
# kill 3276
```

In my case the terminal on which I executed **a.out** was **tty1** and its process id turned out to be **3276**. In your case the terminal name and the process id might be a different number.

If we wish we can abort the execution of the program in the signal handler itself by using the **exit (0)** beyond the **printf( )**.

Note that signals work asynchronously. That is, when a signal is received no matter what our program is doing, the signal handler would immediately get called. Once the execution of the signal handler is over the execution of the program is resumed from the point where it left off when the signal was received.

Handling Multiple Signals

Now that we know how to handle one signal, let us try to handle multiple signals. Here is the program to do this…

```
# include <unistd.h>
# include <sys/types.h>
# include <signal.h>

void inthandler ( int signum )
{
      printf ( "\nSIGINT Received\n" ) ;
}

void termhandler ( int signum )
{
      printf ( "\nSIGTERM Received\n" ) ;
}

void conthandler ( int signum )
{
      printf ( "\nSIGCONT Received\n" ) ;
}

int main( )
{
      signal ( SIGINT, inthandler ) ;
      signal ( SIGTERM, termhandler ) ;
      signal ( SIGCONT, conthandler ) ;

      while ( 1 )
            printf ( "\rProgram Running" ) ;

      return 0 ;
}
```

In this program apart from **SIGINT** we have additionally registered two new signals, namely, **SIGTERM** and **SIGCONT**. The **signal( )** function is called thrice to register a different handler for each of the three signals. After registering the signals we enter a infinite **while** loop to print the 'Program running' message on the screen.

As in the previous program, here too, when we press Ctrl + C the handler for the SIGINT i.e. **inthandler( )** is called. However, when we try to kill the program from the second terminal using the **kill** command the program does not terminate. This is because when the **kill** command is used it sends the running program a **SIGTERM** signal. The default handler for the message terminates the program. Since we have handled this signal ourselves, the handler for **SIGTERM** i.e. **termhandler( )** gets called. As a result the **printf( )** statement in the **termhandler( )** function gets executed and the message 'SIGTERM Received' gets displayed on the screen. Once the execution of **termhandler( )** function is over the program resumes its execution and continues to print 'Program Running'. Then how are we supposed to terminate the program? Simple. Use the following command from the another terminal:

```
kill –SIGKILL 3276
```

As the command indicates, we are trying to send a **SIGKILL** signal to our program. A **SIGKILL** signal terminates the program.

Most signals may be caught by the process, but there are a few signals that the process cannot catch, and they cause the process to terminate. Such signals are often known as un-catchable signals. The **SIGKILL** signal is an un-catchable signal that forcibly terminates the execution of a process.

Note that even if a process attempts to handle the **SIGKILL** signal by registering a handler for it still the control would always land in the default **SIGKILL** handler which would terminate the program.

The **SIGKILL** signal is to be used as a last resort to terminate a program that gets out of control. One such process that makes uses of this signal is a system shutdown process. It first sends a **SIGTERM** signal to all processes, waits for a while, thus giving a 'grace period' to all the running processes. However, after the grace period is over it forcibly terminates all the remaining processes using the **SIGKILL** signal.

That leaves only one question—when does a process receive the **SIGCONT** signal? Let me try to answer this question.

A process under Linux can be suspended using the Ctrl + Z command. The process is stopped but is not terminated, i.e. it is suspended. This gives rise to the un-catchable **SIGSTOP** signal. To resume the execution of the suspended process we can make use of the **fg** (foreground) command. As a result of which the suspended program resumes its execution and receives the **SIGCONT** signal (CONT means continue execution).

Registering a Common Handler

Instead of registering a separate handler for each signal we may decide to handle all signals using a common signal handler. This is shown in the following program:

```
# include <unistd.h>
# include <sys/types.h>
# include <signal.h>

void sighandler ( int signum )
{
    switch ( signum )
    {
        case SIGINT :
```

```
            printf ( "SIGINT Received\n" ) ;
            break ;

    case SIGTERM :
            printf ( "SIGTERM Received\n" ) ;
            break ;

    case SIGCONT :
            printf ( "SIGCONT Received\n" ) ;
            break ;
    }
}

int main( )
{
    signal ( SIGINT,  sighandler ) ;
    signal ( SIGTERM, sighandler ) ;
    signal ( SIGCONT, sighandler ) ;

    while ( 1 )
        printf ( "\rProgram running" ) ;

    return 0 ;
}
```

In this program during each call to the **signal( )** function we have specified the address of a common signal handler named **sighandler( )**. Thus the same signal handler function would get called when one of the three signals are received. This does not lead to a problem since the **sighandler( )** we can figure out inside the signal ID using the first parameter of the function. In our program we have made use of the **switch-case** construct to print a different message for each of the three signals.

Note that we can easily afford to mix the two methods of registering signals in a program. That is, we can register separate signal handlers for some of the signals and a common handler for

some other signals. Registering a common handler makes sense if we want to react to different signals in exactly the same way.

Blocking Signals

Sometimes we may want that flow of execution of a critical/time-critical portion of the program should not be hampered by the occurrence of one or more signals. In such a case we may decide to block the signal. Once we are through with the critical/time-critical code we can unblock the signals(s). Note that if a signal arrives when it is blocked it is simply queued into a signal queue. When the signals are unblocked the process immediately receives all the pending signals one after another. Thus blocking of signals defers the delivery of signals to a process till the execution of some critical/time-critical code is over. Instead of completely ignoring the signals or letting the signals interrupt the execution, it is preferable to block the signals for the moment and deliver them some time later. Let us now write a program to understand signal blocking. Here is the program…

```
# include <unistd.h>
# include <sys/types.h>
# include <signal.h>
# include <stdio.h>

void sighandler ( int signum )
{
    switch ( signum )
    {
        case SIGTERM :
            printf ( "SIGTERM Received\n" ) ;
            break ;

        case SIGINT :
            printf ( "SIGINT Received\n" ) ;
            break ;
```

```
            case SIGCONT :
                printf ( "SIGCONT Received\n" ) ;
                break ;
        }
}

int main( )
{
        char buffer [ 80 ] = "\0" ;
        sigset_t block ;

        signal ( SIGTERM, sighandler ) ;
        signal ( SIGINT,  sighandler ) ;
        signal ( SIGCONT, sighandler ) ;

        sigemptyset ( &block ) ;
        sigaddset ( &block, SIGTERM ) ;
        sigaddset ( &block, SIGINT ) ;

        sigprocmask ( SIG_BLOCK, &block, NULL ) ;

        while ( strcmp ( buffer,"n" ) != 0 )
        {
            printf ( "Enter a String: " ) ;
            gets ( buffer ) ;
            puts ( buffer ) ;
        }

        sigprocmask ( SIG_UNBLOCK, &block, NULL ) ;

        while ( 1 )
            printf ( "\rProgram Running" ) ;

        return 0 ;
}
```

In this program we have registered a common handler for the **SIGINT**, **SIGTERM** and **SIGCONT** signals. Next we want to

repeatedly accept strings in a buffer and display them on the screen till the time the user does not enter an 'n' from the keyboard. Additionally, we want that this activity of receiving input should not be interrupted by the **SIGINT** or the **SIGTERM** signals. However, a **SIGCONT** should be permitted. So before we proceed with the loop we must block the **SIGINT** and **SIGTERM** signals. Once we are through with the loop we must unblock these signals. This blocking and unblocking of signals can be achieved using the **sigprocmask( )** library function.

The first parameter of the **sigprocmask( )** function specifies whether we want to block/unblock a set of signals. The next parameter is the address of a structure (**typedef**ed as **sigset_t**) that describes a set of signals that we want to block/unblock. The last parameter can be either NULL or the address of **sigset_t** type variable which would be set up with the existing set of signals before blocking/unblocking signals.

There are library functions that help us to populate the **sigset_t** structure. The **sigemptyset( )** empties a **sigset_t** variable so that it does not refer to any signals. The only parameter that this function accepts is the address of the **sigset_t** variable. We have used this function to quickly initialize the **sigset_t** variable block to a known empty state. To block the **SIGINT** and **SIGTERM** we have to add the signals to the empty set of signals. This can be achieved using the **sigaddset( )** library function. The first parameter of **sigaddset( )** is the address of the **sigset_t** variable and the second parameter is the ID of the signal that we wish to add to the existing set of signals.

After the loop we have also used an infinite **while** loop to print the 'Program running' message. This is done so that we can easily check that till the time the loop that receives input is not over the program cannot be terminated using Ctrl + C or **kill** command since the signals are blocked. Once the user enters 'n' from the keyboard the execution comes out of the **while** loop and unblocks

the signals. As a result, pending signals, if any, are immediately delivered to the program. So if we press Ctrl + C or use the **kill** command when the execution of the loop that receives input is not over these signals would be kept pending. Once we are through with the loop the signal handlers would be called.

Event Driven programming

Having understood the mechanism of signal processing let us now see how signaling is used by Linux – based libraries to create event driven GUI programs. As you know, in a GUI program events occur typically when we click on the window, type a character, close the window, repaint the window, etc. We have chosen the GTK library version 2.0 to create the GUI applications. Here, GTK stands for Gimp's Tool Kit. Refer Appendix H for installation of this toolkit. Given below is the first program that uses this toolkit to create a window on the screen.

```
/* mywindow.c */
# include <gtk/gtk.h>

int main ( int   argc, char *argv[ ] )
{
      GtkWidget *p ;

      gtk_init ( &argc, &argv ) ;
      p = gtk_window_new ( GTK_WINDOW_TOPLEVEL ) ;
      gtk_window_set_title ( p , "Sample Window" ) ;
      g_signal_connect ( p, "destroy", gtk_main_quit, NULL ) ;
      gtk_widget_set_size_request ( p, 300, 300 ) ;
      gtk_widget_show ( p ) ;
      gtk_main( ) ;

      return 0 ;
}
```

We need to compile this program as follows:

```
gcc  mywindow.c  `pkg-config gtk+-2.0 - -cflags - -libs`
```

Here we are compiling the program 'mywindow.c' and then linking it with the necessary libraries from GTK toolkit. Note the quotes that we have used in the command.

Here is the output of the program…

Figure 21.1

The GTK library provides a large number of functions that makes it very easy for us to create GUI programs. Every window under GTK is known as a widget. To create a simple window we have to carry out the following steps:

(a) Initialize the GTK library with a call to **gtk_init( )** function. This function requires the addresses of the command line arguments received in **main( )**.

(b) Next, call the **gtk_window_new( )** function to create a top level window. The only parameter this function takes is the type of windows to be created. A top level window can be created by specifying the **GTK_WINDOW_TOPLEVEL** value. This call creates a window in memory and returns a pointer to the widget object. The widget object is a structure (**GtkWidget**) variable that stores lots of information including the attributes of window it represents. We have collected this pointer in a **GtkWidget** structure pointer called **p**.

(c) Set the title for the window by making a call to **gtk_window_set_title( )** function. The first parameter of this function is a pointer to the **GtkWidget** structure representing the window for which the title has to be set. The second parameter is a string describing the text to be displayed in the title of the window.

(d) Register a signal handler for the destroy signal. The **destroy** signal is received whenever we try to close the window. The handler for the **destroy** signal should perform clean up activities and then shutdown the application. GTK provides a ready-made function called **gtk_main_quit( )** that does this job. We only need to associate this function with the destroy signal. This can be achieved using the **g_signal_connect( )** function. The first parameter of this function is the pointer to the widget for which destroy signal handler has to be registered. The second parameter is a string that specifies the name of the signal. The third parameter is the address of the signal handler routine. We have not used the fourth parameter.

(e) Resize the window to the desired size using the **gtk_widget_set_size_request( )** function. The second and the

third parameters specify the height and the width of the window respectively.

(f) Display the window on the screen using the function **gtk_widget_show( )**.

(g) Wait in a loop to receive events for the window. This can be accomplished using the **gtk_main( )** function.

How about another program that draws a few shapes in the window? Here is the program…

```
/* myshapes.c */
# include <gtk/gtk.h>

int expose_event ( GtkWidget *widget, GdkEventExpose *event )
{
    GdkGC* p ;
    GdkPoint arr [ 5] = { 250, 150, 250, 300, 300, 350, 400, 300, 320, 190 } ;

    p  = gdk_gc_new ( widget -> window ) ;
    gdk_draw_line ( widget -> window, p, 10, 10, 200, 10 ) ;
    gdk_draw_rectangle ( widget -> window, p, TRUE, 10, 20, 200, 100 ) ;
    gdk_draw_arc ( widget -> window, p, TRUE, 200, 10, 200, 200,
                    2880,  -2880*2 ) ;
    gdk_draw_polygon ( widget -> window, p, TRUE , arr, 5 ) ;
    gdk_gc_unref ( p ) ;

    return TRUE ;
}

int main( int   argc, char *argv[ ] )
{
    GtkWidget *p ;

    gtk_init ( &argc, &argv ) ;
```

```
    p = gtk_window_new ( GTK_WINDOW_TOPLEVEL ) ;
    gtk_window_set_title ( p, "Sample Window" ) ;
    g_signal_connect ( p, "destroy", gtk_main_quit, NULL ) ;
    g_signal_connect ( p , "expose_event", expose_event, NULL ) ;
    gtk_widget_set_size_request ( p,  500,  500 ) ;
    gtk_widget_show ( p ) ;
    gtk_main( ) ;

    return 0 ;
}
```

Given below is the output of the program.

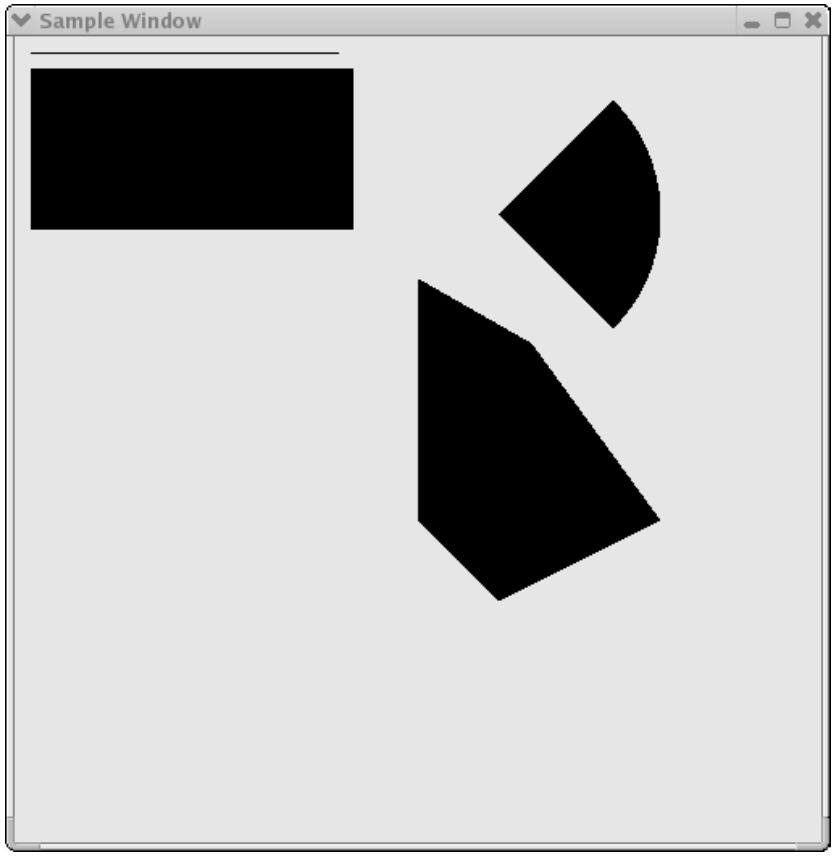

Figure 21.2

This program is similar to the first one. The only difference is that in addition to the destroy signal we have registered a signal handler for the **expose_event** using the **g_signal_connect( )** function. This signal is sent to our process whenever the window needs to be redrawn. By writing the code for drawing shapes in the handler for this signal we are assured that the drawing would never vanish if the windows is dragged outside the screen and then brought back in, or some other window uncovers a portion of our window which was previously overlapped, and so on. This is

because a **expose_event** signal would be sent to our application which would immediately redraw the shapes in our window.

The way in Windows we have a device context, under Linux we have a graphics context. In order to draw in the window we need to obtain a graphics context for the window using the **gdk_gc_new( )** function. This function returns a pointer to the graphics context structure. This pointer must be passed to the drawing functions like **gdk_draw_line( )**, **gdk_draw_rectangle()**, **gdk_draw_arc( )**, **gdk_draw_polygon( )**, etc. Once we are through with drawing we should release the graphics context using the **gdk_gc_unref( )** function.

Where Do You Go From Here

You have now understood signal processing, the heart of programming under Linux. With that knowledge under your belt you are now capable of exploring the vast world of Linux on your own. Complete Linux programming deserves a book on its own. Idea here was to raise the hood and show you what lies underneath it. I am sure that if you have taken a good look at it you can try the rest yourselves. Good luck!

Summary

(a) Programs can communicate with the Linux OS using library functions.
(b) The Linux OS communicates with a program by means of signals.
(c) The interrupt signal (**SIGINT**) is sent by the kernel to our program when we press Ctrl + C.
(d) A term signal (**SIGTERM**) is sent to the program when we use the **kill** command.
(e) A process cannot handle an un-catchable signal.
(f) The **kill –SIGKILL** variation of the **kill** command generates an un-catchable **SIGKILL** signal that terminates a process.

(g) A process can block a signal or a set of signals using the **sigprocmask( )** function.
(h) Blocked signals are delivered to the process when the signals are unblocked.
(i) A **SIGSTOP** signal is generated when we press Ctrl + Z.
(j) A **SIGSTOP** signal is un-catchable signal.
(k) A suspended process can be resumed using the **fg** command.
(l) A process receives the **SIGCONT** signal when it resumes execution.
(m) In GTK, the **g_signal_connect( )** function can be used to connect a function with an event.

Exercise

[A] State True or False:

(a) All signals registered signals must have a separate signal handler.
(b) Blocked signals are ignored by a process.
(c) Only one signal can be blocked at a time.
(d) Blocked signals are ignored once the signals are unblocked.
(e) If our signal handler gets called the default signal handler automatically gets called.
(f) **gtk_main( )** function makes uses of a loop to prevent the termination of the program.
(g) Multiple signals can be registered at a time using a single call to **signal( )** function.
(h) The **sigprocmask( )** function can block as well as unblock signals.

[B] Answer the following:

(a) How does the Linux OS know if we have registered a signal or not?

(b) What happens when we register a handler for a signal?

(c) Write a program to verify that **SIGSTOP** and **SIGKILL** signals are un-catchable signals.

(d) Write a program to handle the **SIGINT** and **SIGTERM** signals. From inside the handler for **SIGINT** signal write an infinite loop to print the message 'Processing Signal'. Run the program and make use of Ctrl + C more than once. Run the program once again and press Ctrl + C once then use the **kill** command. What are your observations?

(e) Write a program that blocks the **SIGTERM** signal during execution of the **SIGINT** signal.

A Precedence Table

Description	Operator	Associativity
Function expression	( )	Left to Right
Array Expression	[ ]	Left to Right
Structure operator	->	Left to Right
Structure operator	.	Left to Right
Unary minus	-	Right to left
Increment/Decrement	++ --	Right to Left
One's compliment	~	Right to left
Negation	!	Right to Left
Address of	&	Right to left
Value of address	*	Right to left
Type cast	(type)	Right to left
Size in bytes	sizeof	Right to left
Multiplication	*	Left to right
Division	/	Left to right
Modulus	%	Left to right
Addition	+	Left to right
Subtraction	-	Left to right
Left shift	<<	Left to right
Right shift	>>	Left to right
Less than	<	Left to right
Less than or equal to	<=	Left to right
Greater than	>	Left to right
Greater than or equal to	>=	Left to right
Equal to	==	Left to right
Not equal to	!=	Left to right

Continued…

Continued…

Description	**Operator**	**Associativity**
Bitwise AND	&	Left to right
Bitwise exclusive OR	^	Left to right
Bitwise inclusive OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	? :	Right to left
Assignment	=	Right to left
	*= /= %=	Right to left
	+= -= &=	Right to left
	^= |=	Right to left
	<<= >>=	Right to left
Comma	,	Right to left

Figure A1.1

B Standard Library Functions

- Standard Library Functions
- Arithmetic Functions
- Data Conversion Functions
- Character Classification Functions
- String Manipulation Functions
- Searching and Sorting Functions
- I/O Functions
- File Handling Functions
- Directory Control Functions
- Buffer Manipulation Functions
- Disk I/O Functions
- Memory Allocation Functions
- Process Control Functions
- Graphics Functions
- Time Related Functions
- Miscellaneous Functions
- DOS Interface Functions

Let alone discussing each standard library function in detail, even a complete list of these functions would occupy scores of pages. However, this book would be incomplete if it has nothing to say about standard library functions. I have tried to reach a compromise and have given a list of standard library functions that are more popularly used so that you know what to search for in the manual. An excellent book dedicated totally to standard library functions is Waite group's, Turbo C Bible, written by Nabjyoti Barkakti.

Following is the list of selected standard library functions. The functions have been classified into broad categories.

Arithmetic Functions

Function	Use
abs	Returns the absolute value of an integer
cos	Calculates cosine
cosh	Calculates hyperbolic cosine
exp	Raises the exponential e to the x^{th} power
fabs	Finds absolute value
floor	Finds largest integer less than or equal to argument
fmod	Finds floating-point remainder
hypot	Calculates hypotenuse of right triangle
log	Calculates natural logarithm
log10	Calculates base 10 logarithm
modf	Breaks down argument into integer and fractional parts
pow	Calculates a value raised to a power
sin	Calculates sine
sinh	Calculates hyperbolic sine
sqrt	Finds square root
tan	Calculates tangent
tanh	Calculates hyperbolic tangent

Data Conversion Functions

Function	Use
atof	Converts string to float
atoi	Converts string to int
atol	Converts string to long
ecvt	Converts double to string
fcvt	Converts double to string
gcvt	Converts double to string
itoa	Converts int to string
ltoa	Converts long to string
strtod	Converts string to double
strtol	Converts string to long integer
strtoul	Converts string to an unsigned long integer
ultoa	Converts unsigned long to string

Character classification Functions

Function	Use
isalnum	Tests for alphanumeric character
isalpha	Tests for alphabetic character
isdigit	Tests for decimal digit
islower	Tests for lowercase character
isspace	Tests for white space character
isupper	Tests for uppercase character
isxdigit	Tests for hexadecimal digit
tolower	Tests character and converts to lowercase if uppercase
toupper	Tests character and converts to uppercase if lowercase

String Manipulation Functions

Function	Use
strcat	Appends one string to another
strchr	Finds first occurrence of a given character in a string
strcmp	Compares two strings
strcmpi	Compares two strings without regard to case
strcpy	Copies one string to another
strdup	Duplicates a string
stricmp	Compares two strings without regard to case (identical to strcmpi)
strlen	Finds length of a string
strlwr	Converts a string to lowercase
strncat	Appends a portion of one string to another
strncmp	Compares a portion of one string with portion of another string
strncpy	Copies a given number of characters of one string to another
strnicmp	Compares a portion of one string with a portion of another without regard to case
strrchr	Finds last occurrence of a given character in a string
strrev	Reverses a string
strset	Sets all characters in a string to a given character
strstr	Finds first occurrence of a given string in another string
strupr	Converts a string to uppercase

Searching and Sorting Functions

Function	Use
bsearch	Performs binary search
lfind	Performs linear search for a given value
qsort	Performs quick sort

I/O Functions

Function	Use
Close	Closes a file
fclose	Closes a file
feof	Detects end-of-file
fgetc	Reads a character from a file
fgetchar	Reads a character from keyboard (function version)
fgets	Reads a string from a file
fopen	Opens a file
fprintf	Writes formatted data to a file
fputc	Writes a character to a file
fputchar	Writes a character to screen (function version)
fputs	Writes a string to a file
fscanf	Reads formatted data from a file
fseek	Repositions file pointer to given location
ftell	Gets current file pointer position
getc	Reads a character from a file (macro version)
getch	Reads a character from the keyboard
getche	Reads a character from keyboard and echoes it
getchar	Reads a character from keyboard (macro version)
gets	Reads a line from keyboard
inport	Reads a two-byte word from the specified I/O port
inportb	Reads one byte from the specified I/O port
kbhit	Checks for a keystroke at the keyboard
lseek	Repositions file pointer to a given location
open	Opens a file
outport	Writes a two-byte word to the specified I/O port
outportb	Writes one byte to the specified I/O port
printf	Writes formatted data to screen
putc	Writes a character to a file (macro version)
putch	Writes a character to the screen
putchar	Writes a character to screen (macro version)
puts	Writes a line to file
read	Reads data from a file

rewind	Repositions file pointer to beginning of a file
scanf	Reads formatted data from keyboard
sscanf	Reads formatted input from a string
sprintf	Writes formatted output to a string
tell	Gets current file pointer position
write	Writes data to a file

File Handling Functions

Function	Use
remove	Deletes file
rename	Renames file
unlink	Deletes file

Directory Control Functions

Function	Use
chdir	Changes current working directory
getcwd	Gets current working directory
fnsplit	Splits a full path name into its components
findfirst	Searches a disk directory
findnext	Continues *findfirst* search
mkdir	Makes a new directory
rmdir	Removes a directory

Buffer Manipulation Functions

Function	Use
memchr	Returns a pointer to the first occurrence, within a specified number of characters, of a given character in the buffer
memcmp	Compares a specified number of characters from two buffers

memcpy	Copies a specified number of characters from one buffer to another
memicmp	Compares a specified number of characters from two buffers without regard to the case of the characters
memmove	Copies a specified number of characters from one buffer to another
memset	Uses a given character to initialize a specified number of bytes in the buffer

Disk I/O Functions

Function	Use
absread	Reads absolute disk sectors
abswrite	Writes absolute disk sectors
biosdisk	Performs BIOS disk services
getdisk	Gets current drive number
setdisk	Sets current disk drive

Memory Allocation Functions

Function	Use
calloc	Allocates a block of memory
farmalloc	Allocates memory from far heap
farfree	Frees a block from far heap
free	Frees a block allocated with *malloc*
malloc	Allocates a block of memory
realloc	Reallocates a block of memory

Process Control Functions

Function	Use
abort	Aborts a process
atexit	Executes function at program termination

execl	Executes child process with argument list
exit	Terminates the process
spawnl	Executes child process with argument list
spawnlp	Executes child process using PATH variable and argument list
system	Executes an MS-DOS command

Graphics Functions

Function	Use
arc	Draws an arc
ellipse	Draws an ellipse
floodfill	Fills an area of the screen with the current color
getimage	Stores a screen image in memory
getlinestyle	Obtains the current line style
getpixel	Obtains the pixel's value
lineto	Draws a line from the current graphic output position to the specified point
moveto	Moves the current graphic output position to a specified point
pieslice	Draws a pie-slice-shaped figure
putimage	Retrieves an image from memory and displays it
rectangle	Draws a rectangle
setcolor	Sets the current color
setlinestyle	Sets the current line style
putpixel	Plots a pixel at a specified point
setviewport	Limits graphic output and positions the logical origin within the limited area

Time Related Functions

Function	Use
clock	Returns the elapsed CPU time for a process
difftime	Computes the difference between two times

ftime	Gets current system time as structure
strdate	Returns the current system date as a string
strtime	Returns the current system time as a string
time	Gets current system time as long integer
setdate	Sets DOS date
getdate	Gets system date

Miscellaneous Functions

Function	Use
delay	Suspends execution for an interval (milliseconds)
getenv	Gets value of environment variable
getpsp	Gets the Program Segment Prefix
perror	Prints error message
putenv	Adds or modifies value of environment variable
random	Generates random numbers
randomize	Initializes random number generation with a random value based on time
sound	Turns PC speaker on at specified frequency
nosound	Turns PC speaker off

DOS Interface Functions

Function	Use
FP_OFF	Returns offset portion of a far pointer
FP_SEG	Returns segment portion of a far pointer
getvect	Gets the current value of the specified interrupt vector
keep	Installs terminate-and-stay-resident (TSR) programs
int86	Issues interrupts
int86x	Issues interrupts with segment register values
intdos	Issues interrupt 21h using registers other than DX and AL
intdosx	Issues interrupt 21h using segment register values
MK_FP	Makes a far pointer

segread	Returns current values of segment registers
setvect	Sets the current value of the specified interrupt vector

C Chasing The Bugs

C programmers are great innovators of our times. Unhappily, among their most enduring accomplishments are several new techniques for wasting time. There is no shortage of horror stories about programs that took twenty times to 'debug' as they did to 'write'. And one hears again and again about programs that had to be rewritten all over again because the bugs present in it could not be located. A typical C programmer's 'morning after' is red eyes, blue face and a pile of crumpled printouts and dozens of reference books all over the floor. Bugs are C programmer's birthright. But how do we chase them away. No sure-shot way for that. I thought if I make a list of more common programming mistakes it might be of help. They are not arranged in any particular order. But as you would realize surely a great help!

[1] Omitting the ampersand before the variables used in **scanf( )**.

For example,

```
int  choice ;
scanf ( "%d", choice ) ;
```

Here, the **&** before the variable choice is missing. Another common mistake with **scanf( )** is to give blanks either just before the format string or immediately after the format string as in,

```
int  choice ;
scanf ( " %d ", choice ) ;
```

Note that this is not a mistake, but till you don't understand **scanf( )** thoroughly, this is going to cause trouble. Safety is in eliminating the blanks. Thus, the correct form would be,

```
int  choice ;
scanf ( "%d", &choice ) ;
```

[2] Using the operator = instead of the operator = =.

What do you think will be the output of the following program:

```
main( )
{
    int  i = 10 ;

    while ( i = 10 )
    {
        printf ( "got to get out" ) ;
        i++ ;
    }
}
```

At first glance it appears the message will be printed once and the control will come out of the loop since **i** becomes 11. But, actually we have fallen in an indefinite loop. This is because the = used in the condition always assigns the value 10 to **i**, and since **i** is non-zero the condition is satisfied and the body of the loop is executed over and over again.

[3] Ending a loop with a semicolon.

Observe the following program.

```
main( )
{
    int  j = 1 ;

    while ( j <= 100 ) ;
    {
        printf ( "\nCompguard" ) ;
        j++ ;
    }
}
```

Inadvertently, we have fallen in an indefinite loop. Cause is the semicolon after **while**. This in effect makes the compiler feel that you wanted the loop to work in the following manner:

```
while ( j <= 100 ) ;
```

This is an indefinite loop since **j** never gets incremented and hence eternally remains less that 100.

[4] Omitting the **break** statement at the end of a **case** in a **switch** statement.

Remember that if a **break** is not included at the end of a **case**, then execution will continue into the next **case**.

```
main( )
{
    int  ch = 1 ;

    switch ( ch )
    {
        case 1 :
            printf ( "\nGoodbye" ) ;
        case 2 :
            printf ( "\nLieutenant" ) ;
    }
}
```

Here, since the **break** has not been given after the **printf( )** in **case 1**, the control runs into **case 2** and executes the second **printf( )** as well.

However, this sometimes turns out to be a blessing in disguise. Especially, in cases when we are checking whether the value of a variable equals a capital letter or a small case

letter. This example has been succinctly explained in Chapter 4.

[5] Using **continue** in a **switch**.

It is a common error to believe that the way the keyword **break** is used with loops and a **switch**; similarly the keyword **continue** can also be used with them. Remember that **continue** works only with loops, never with a **switch**.

[6] A mismatch in the number, type and order of actual and formal arguments.

```
yr = romanise ( year, 1000, 'm' ) ;
```

Here, three arguments in the order **int**, **int** and **char** are being passed to **romanise( )**. When **romanise( )** receives these arguments into formal arguments they must be received in the same order. A careless mismatch might give strange results.

[7] Omitting provisions for returning a non-integer value from a function.

If we make the following function call,

```
area = area_circle ( 1.5 ) ;
```

then while defining **area_circle( )** function later in the program, care should be taken to make it capable of returning a floating point value. Note that unless otherwise mentioned the compiler would assume that this function returns a value of the type **int**.

[8] Inserting a semicolon at the end of a macro definition.

How do you recognize a C programmer? Ask him to write a paragraph in English and watch whether he ends each sentence with a semicolon. This usually happens because a C programmer becomes habitual to ending all statements with a semicolon. However, a semicolon at the end of a macro definition might create a problem. For example,

```
#define UPPER 25 ;
```

would lead to a syntax error if used in an expression such as

```
if ( counter == UPPER )
```

This is because on preprocessing, the **if** statement would take the form

```
if ( counter == 25 )
```

[9] Omitting parentheses around a macro expansion.

```
#define SQR(x) x * x
main( )
{
    int  a ;

    a = 25 / SQR ( 5 ) ;
    printf ( "\n%d", a ) ;
}
```

In this example we expect the value of **a** to be 1, whereas it turns out to be 25. This so happens because on preprocessing the arithmetic statement takes the following form:

```
a = 25 / 5 * 5 ;
```

[10] Leaving a blank space between the macro template and the macro expansion.

```
#define ABS (a) ( a = 0  ? a : -a )
```

Here, the space between **ABS** and **(a)** makes the preprocessor believe that you want to expand **ABS** into **(a)**, which is certainly not what you want.

[11] Using an expression that has side effects in a macro call.

```
#define SUM ( a ) ( a + a )
main( )
{
    int  w, b = 5 ;

    w = SUM( b++ ) ;
    printf ( "\n%d", w ) ;
}
```

On preprocessing, the macro would be expanded to,

```
w = ( b++ ) + ( b++ ) ;
```

If you are wanting to first get sum of 5 and 5 and then increment **b** to 6, that would not happen using the above macro definition.

[12] Confusing a character constant and a character string.

In the statement

```
ch = 'z' ;
```

a single character is assigned to **ch**. In the statement

```
ch = "z" ;
```

a pointer to the character string "a" is assigned to **ch**.

Note that in the first case, the declaration of **ch** would be,

```
char  ch ;
```

whereas in the second case it would be,

```
char  *ch ;
```

[13] Forgetting the bounds of an array.

```
main( )
{
    int  num[50], i ;

    for ( i = 1 ; i <= 50 ; i++ )
        num[i] = i * i ;
}
```

Here, in the array **num** there is no such element as **num[50]**, since array counting begins with 0 and not 1. Compiler would not give a warning if our program exceeds the bounds. If not taken care of, in extreme cases the above code might even hang the computer.

[14] Forgetting to reserve an extra location in a character array for the null terminator.

Remember each character array ends with a '\0', therefore its dimension should be declared big enough to hold the normal characters as well as the '\0'.

For example, the dimension of the array **word[ ]** should be 9 if a string "Jamboree" is to be stored in it.

[15] Confusing the precedences of the various operators.

```
main( )
{
    char  ch ;
    FILE  *fp ;

    fp = fopen ( "text.c", "r" ) ;

    while ( ch = getc ( fp ) != EOF )
          putch ( ch ) ;

    fclose ( fp ) ;
}
```

Here, the value returned by **getc( )** will be first compared with EOF, since **!=** has a higher priority than **=**. As a result, the value that is assigned to **ch** will be the true/false result of the test—1 if the value returned by **getc( )** is not equal to **EOF**, and 0 otherwise. The correct form of the above **while** would be,

```
while ( ( ch = getc ( fp ) ) != EOF )
    putch ( ch ) ;
```

[16] Confusing the operator -> with the operator **.** while referring to a structure element.

Remember, on the left of the operator **.** only a structure variable can occur, whereas on the left of the operator -> only a pointer to a structure can occur. Following example demonstrates this.

```
main( )
```

```
{
    struct emp
    {
        char  name[35] ;
        int  age ;
    } ;
    struct emp e = { "Dubhashi", 40 } ;
    struct emp *ee ;

    printf ( "\n%d", e.age ) ;
    ee = &e ;
    printf ( "\n%d", ee->>age ) ;
}
```

[17] Forgetting to use the **far** keyword for referring memory locations beyond the data segment.

```
main( )
{
    unsigned int  *s ;

    s = 0x413 ;
    printf ( "\n%d", *s ) ;
}
```

Here, it is necessary to use the keyword **far** in the declaration of variable **s**, since the address that we are storing in **s** (0x413) is a address of location present in BIOS Data Area, which is far away from the data segment. Thus, the correct declaration would look like,

```
unsigned int far  *s ;
```

The far pointers are 4-byte pointers and are specific to DOS. Under Windows every pointer is 4-byte pointer.

[18] Exceeding the range of integers and chars.

```
main( )
{
    char  ch ;

    for ( ch = 0 ; ch <= 255 ; ch++ )
        printf ( "\n%c %d", ch, ch ) ;
}
```

Can you believe that this is an indefinite loop? Probably, a closer look would confirm it. Reason is, **ch** has been declared as a **char** and the valid range of **char** constant is -128 to +127. Hence, the moment **ch** tries to become 128 (through **ch**++), the value of character range is exceeded, therefore the first number from the negative side of the range, -128, gets assigned to **ch**. Naturally the condition is satisfied and the control remains within the loop externally.

C Creating Libraries

In Chapter 5 we saw how to add/delete functions to/from existing libraries. At times we may want to create our own library of functions. Here we would assume that we wish to create a library containing the functions **factorial( )**, **prime( )** and **fibonacci( )**. As their names suggest, **factorial( )** calculates and returns the factorial value of the integer passed to it, **prime( )** reports whether the number passed to it is a prime number or not and **fibonacci( )** prints the first **n** terms of the Fibonacci series, where **n** is the number passed to it. Here are the steps that need to be carried out to create this library. Note that these steps are specific to Turbo C/C++ compiler and would vary for other compilers.

(a) Define the functions **factorial( )**, **prime( )** and **fibonacci( )** in a file, say 'myfuncs.c'. Do not define **main( )** in this file.

(b) Create a file 'myfuncs.h' and declare the prototypes of **factorial( )**, **prime( )** and **fibonacci( )** in it as shown below:

```
int factorial ( int ) ;
int prime ( int ) ;
void fibonacci ( int ) ;
```

(c) From the Options menu select the menu-item 'Application'. From the dialog that pops us select the option 'Library'. Select OK.

(d) Compile the program using Alt F9. This would create the library file called 'myfuncs.lib'.

That's it. The library now stands created. Now we have to use the functions defined in this library. Here is how it can be done.

(a) Create a file, say 'sample.c' and type the following code in it.

```
#include "myfuncs.h"
main( )
```

```
{
    int  f, result ;
    f = factorial ( 5 ) ;
    result = prime ( 13 ) ;
    fibonacci ( 6 ) ;
    printf ( "\n%d %d", f, result ) ;
}
```

Note that the file 'myfuncs.h' should be in the same directory as the file 'sample.c'. If not, then while including 'myfuncs.h' mention the appropriate path.

(b) Go to the 'Project' menu and select 'Open Project…' option. On doing so a dialog would pop up. Give the name of the project, say 'sample.prj' and select OK.

(c) From the 'Project' menu select 'Add Item'. On doing so a file dialog would appear. Select the file 'sample.c' and then select 'Add'. Also add the file 'myfuncs.lib' in the same manner. Finally select 'Done'.

(d) Compile and execute the project using Ctrl F9.

D Hexadecimal Numbering

- Numbering Systems
- Relation Between Binary and Hex

While working with computers we are often required to use hexadecimal numbers. The reason for this is—everything a computer does is based on binary numbers, and hexadecimal notation is a convenient way of expressing binary numbers. Before justifying this statement let us first discuss what numbering systems are, why computers use binary numbering system, how binary and hexadecimal numbering systems are related and how to use hexadecimal numbering system in everyday life.

Numbering Systems

When we talk about different numbering systems we are really talking about the base of the numbering system. For example, binary numbering system has base 2 and hexadecimal numbering system has base 16, just the way decimal numbering system has base 10. What in fact is the 'base' of the numbering system? Base represents number of digits you can use before you run out of digits. For example, in decimal numbering system, when we have used digits from 0 to 9, we run out of digits. That's the time we put a 1 in the column to the left - the ten's column - and start again in the one's column with 0, as shown below:

```
0
1
2
3
4
5
6
7
8
9     last available digit
10    start using a new column
11
12
13
```